

SOC/L1 -11.2. Intro to Malware Analysis

Task 1 Introduction:

Every once in a while, when working as a SOC analyst, you will encounter content (a file or traffic) that appears suspicious, and you will need to determine whether it is malicious.

Task 2 Malware Analysis:

The word malware is derived from the term MALicious softWARE. Malware analysis is an important skill. Malware Analysis is performed by the following people in the Security Industry:

- Security Operations teams analyse malware to write detections for malicious activity in their networks.
- Incident Response** teams analyse malware to determine what damage has been done to an environment and to remediate and revert that damage.
- Threat Hunt teams analyse malware to identify IOCs, which they use to hunt for malware in a network.
- Malware Researchers in security product vendor teams analyse malware to add detections for it in their security products.
- Threat Research teams in Vendors like Microsoft and Google analyse malware to discover the vulnerabilities exploited and add more security features to the /applications.

Task 3 Techniques of malware analysis:

Most of the time, you will have an executable file (also called a binary or a PE file. PE stands for Portable Executable), a malicious document file, or a Network Packet Capture (pcap). The Portable Executable is the most prevalent type of file analysed while performing Malware Analysis.

Static Analysis: When malware is analysed without being executed, it is called Static Analysis. In this case, the different properties of the PE file are

analysed without running it.

Dynamic Analysis: Dynamic analysis techniques include running the malware in a VM, either in a manual fashion with tools installed to monitor the malware's activity or in the form of sandboxes that perform this task automatically.

Remnux: (Reverse Engineering Malware Linux) is a Linux distribution purpose-built for malware analysis.

cd Desktop/Samples file WannaCry (determined the type of a file by inspecting its content rather than relying on its extension.)

Task 4 Basic Static Analysis

Examining Strings

```
user@machine$ file wannacry
wannacry: PE32 executable (GUI) Intel 80386, for MS Windows
user@machine$
```

```
man file or file --help
man strings or strings --help
"provides detailed documentation for commands"
```

strings <filename>

(extracted human-readable text from binary or non-text files)

```
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ strings wannacry
> result.txt
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ vim result.txt
'We converted the result of the wannacry file, into a text file
from which we obtained the string'
```

Calculating Hashes

```
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ md5sum wannacry
84c82835a5d21bbcf75a61706d8ab549  wannacry
```

'We took the WannaCry sample and had it investigated on Virus Total.'

Task 5 The PE file Header

PE Header: A file format for executables, object code, DLLs, FON Font files, and others used in 32-bit and 64-bit versions of Windows operating systems. The PE format is a data structure that encapsulates the information necessary for the Windows OS to manage the wrapped executable code.

The PE File Header(opens in new tab) contains the metadata about a Portable Executable file.

user@machine\$ pecheck WannaCry

(pecheck is designed to verify and create PE checksums for Portable Executable (PE) files.)

```
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ pecheck redline
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ pecheck redline >
red_result.txt
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ grep RegOpenKeyEx
W red_result.txt
ubuntu@ip-10-112-169-186:~/Desktop/Samples$ pe-tree redline
'The pe-tree command is part of the PE Tree tool, which is a
Python module for viewing Portable Executable (PE) files in a
tree-view using pefile and PyQt5'
```

Task 6 Basic Dynamic Analysis

Dynamic analysis requires running live malware samples, which can be destructive. It is highly recommended that you perform malware analysis on an isolated Virtual Machine.

Sandbox: Hardware or software dedicated for isolating untrusted applications or services from critical system resources and other programs to prevent harmful actions or malware from negatively affecting the system.

For malware analysis using sandboxes, the following considerations make the malware analysis effective:

Virtual Machine mimicking the actual target environment of the malware sample

Ability to take snapshots and revert to a clean state

OS monitoring software, such as Procmon, ProcExplorer, Regshot, etc.

Network monitoring software, such as Wireshark, tcpdump, etc.

Control over the network through a dummy DNS server and web server.

A mechanism to move analysis logs and malware samples in and out of the Virtual Machine without compromising the host (Be careful with this one).

Open Source Sandboxes

Cuckoo's Sandbox: Cuckoo's sandbox([opens in new tab](#)) is the most widely known sandbox in the malware analysis community. It was developed as part of a Google Summer of Code project in 2010. It is an open-source project that you will often see deployed in SOC environments and with enthusiasts' home labs.

Cuckoo's Sandbox: Cuckoo's sandbox([opens in new tab](#)) is the most widely known sandbox in the malware analysis community. It was developed as part of a Google Summer of Code project in 2010. It is an open-source project that you will often see deployed in SOC environments and with enthusiasts' home labs.

Online Sandboxes: Setting up and maintaining a sandbox can be a time-consuming task. Keeping that in view, online sandboxes can be of great help. Some of the most commonly used online sandboxes are as follows:

Online Cuckoo Sandbox

Any.run

Intezer

Hybrid Analysis

Answer the questions below:

Check the hash of the sample 'redline' on Hybrid Analysis from AttackBox or your device. In the process tree, which is the first process launched by the sample?

setup_installer.exe

In the process tree, there are two Windows utilities abused by the malware.
What are the names of the two utilities?

(Format: utility1.exe and utility2.exe) cmd.exe and powershell.exe

Task 7 Anti-analysis techniques

Malware authors often use packing and obfuscation to make an analyst's life difficult. A packer obfuscates, compresses, or encrypts the contents of malware. These techniques make it difficult to analyse malware statically

```
user@machine$ strings zmsuz3pinwl
```

```
user@machine$ pecheck zmsuz3pinwl
```

(string search against the file named zmsuz3pinwl in the Samples folder in the attached VM. We will notice that this sample contains mainly garbage strings that don't provide much value to us. Like: Suspicious flags set for section 2. Both IMAGE_SCN_MEM_WRITE)

Sandbox evasion

Some of these techniques are as follows:

Long sleep calls: Malware authors know that sandboxes run for a limited time. Therefore, they program the malware not to perform any activity for a long time after execution. This is often accomplished through long sleep calls. The purpose of this technique is to time out the sandbox.

User activity detection: Some malware samples will wait for user activity before performing malicious activity. The premise of this technique is that there will be no user in a sandbox. Therefore, there will be no mouse movement or typing on the keyboard. Advanced malware also detects patterns in mouse movements that are often used in automated sandboxes. This technique is designed to bypass automated sandbox detection.

Footprinting user activity: Some malware checks for user files or activity, such as whether there are any files in the MS Office history or internet browsing history. If no or little activity is found, the malware will consider the machine as a sandbox and quit.

Detecting VMs: Sandboxes run on virtual machines. Virtual machines leave artefacts that can be identified by malware. For example, some drivers installed in VMs being run on VMWare or Virtualbox give away the fact that the machine

is a VM. Malware authors often associate VMs with sandboxes and would terminate the malware if a VM is detected.